

Experiment 2: 8-Queen Problem

Aim

To solve the 8-Queen problem using Python.

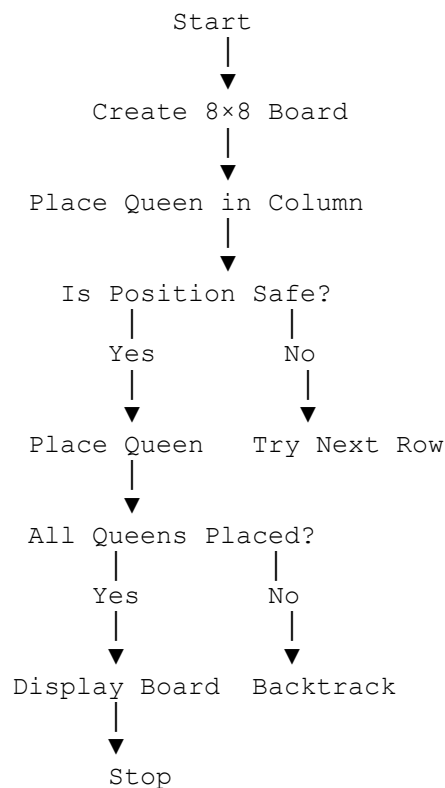
Objective

To place 8 queens on an 8×8 chessboard such that no two queens attack each other.

Algorithm

1. Create an 8×8 chessboard.
2. Place a queen in the first column.
3. Check whether the position is safe.
4. If safe, place the queen and move to the next column.
5. If no safe position exists, backtrack.
6. Repeat until all 8 queens are placed.
7. Display the final board.

Flowchart



Python Program

```
N = 8

board = [[0] * N for _ in range(N)]

def safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col == N:
        return True

    for i in range(N):
        if safe(i, col):
            board[i][col] = 1

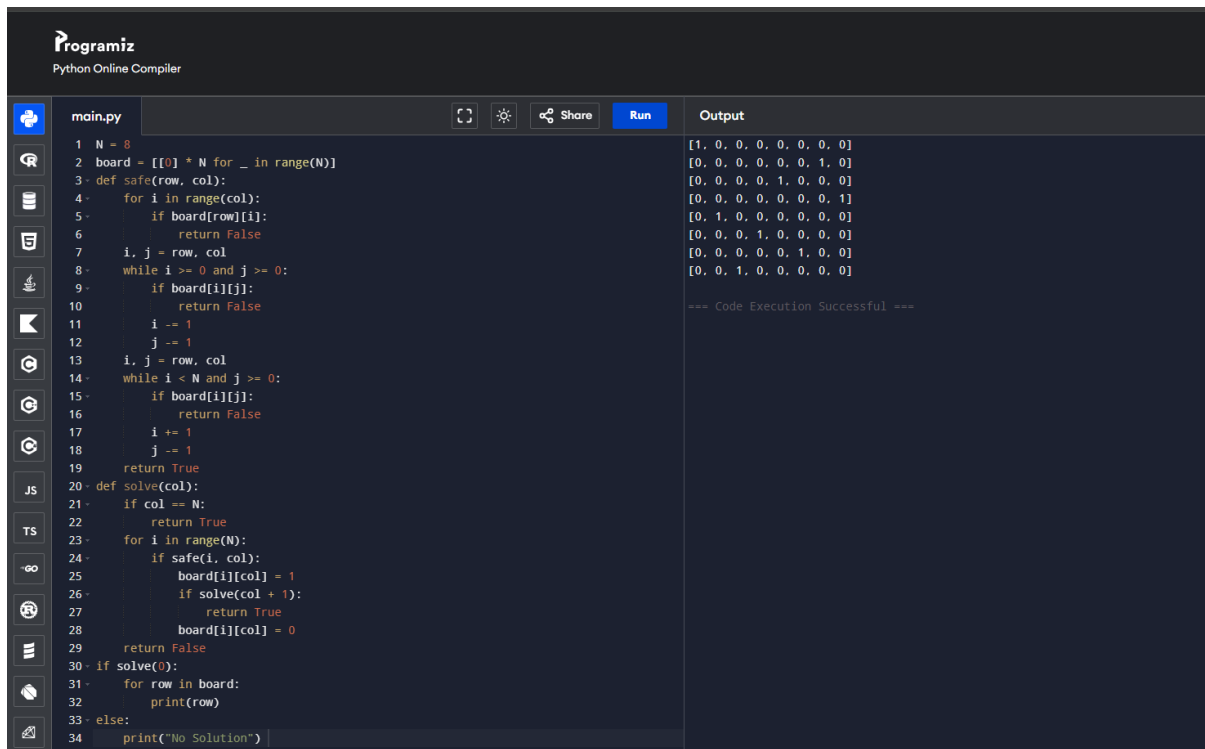
            if solve(col + 1):
                return True

            board[i][col] = 0

    return False

if solve(0):
    for row in board:
        print(row)
else:
    print("No Solution")
```

Sample Output



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for solving the 8-Queen problem using backtracking. The script defines a `safe` function to check for conflicts, a `solve` function to recursively place queens, and a main execution block that prints the solution or reports no solution. The output panel on the right displays the resulting 8x8 chessboard configuration, where each row contains exactly one queen (represented by 1) and all other positions are empty (represented by 0). The output also includes a success message: "=== Code Execution Successful ===".

```
1 N = 8
2 board = [[0] * N for _ in range(N)]
3 def safe(row, col):
4     for i in range(col):
5         if board[row][i]:
6             return False
7     i, j = row, col
8     while i >= 0 and j >= 0:
9         if board[i][j]:
10            return False
11        i -= 1
12        j -= 1
13    i, j = row, col
14    while i < N and j >= 0:
15        if board[i][j]:
16            return False
17        i += 1
18        j -= 1
19    return True
20 def solve(col):
21     if col == N:
22         return True
23     for i in range(N):
24         if safe(i, col):
25             board[i][col] = 1
26             if solve(col + 1):
27                 return True
28             board[i][col] = 0
29     return False
30 if solve(0):
31     for row in board:
32         print(row)
33 else:
34     print("No Solution")
```

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]

=== Code Execution Successful ===
```

Here:

- 1 → Queen
- 0 → Empty position

Result

Thus, the 8-Queen problem was solved successfully using Python by placing all eight queens safely on the chessboard.

Conclusion

The program uses the **backtracking algorithm** to place queens one by one. If a conflict occurs, it backtracks and tries another position until a valid arrangement of all 8 queens is found.